

# Object-Oriented Programming

## Lecture 4: OOP - 1

**Prof. Jaehyung Seo**

(seojae777@konkuk.ac.kr)

**Language Intelligence & Representation Lab.**

# The "Is-A" Relationship

## Inheritance as a Design Principle

- Establishes an "is-a" relationship: A Car is a Vehicle, a Motorcycle is a Vehicle.

## Superclass (Blueprint)

- A generic Vehicle blueprint defines common traits: engine, wheels.

## Subclass (Specialization)

- A Car adds four doors; a Motorcycle adds handlebars.

## Key Benefit

- Abstract common details into superclasses, store specific ones in subclasses → **eliminates duplicate code.**

# Implementing Basic Inheritance

## Code Snippet:

```
class Contact:
    all_contacts = []
    def __init__(self, name: str, email: str):
        self.name = name
        self.email = email
        Contact.all_contacts.append(self)

# Supplier inherits from Contact
class Supplier(Contact):
    def order(self, order_request: str):
        print(f"Sending '{order_request}' to {self.name}")
```

# Extending Built-in Types

## Definition

- Python allows inheritance directly from core data structures (e.g., `list`, `dict`, `str`) to add custom behaviors.

## Example Context

- Imagine needing a list that can `search itself`. Instead of building a list from scratch, we subclass Python's highly optimized `list`.

## Key Benefit

- Leverage Python's C-level optimizations while injecting your domain logic.

# Subclassing Python's list

## Code Snippet:

```
class ContactList(list):  
    def search(self, name: str) -> list:  
        # 'self' behaves exactly like a standard Python list  
        return [c for c in self if name in c.name]  
  
# Usage:  
contacts = ContactList()  
contacts.append(Contact("John", "j@e.com"))  
matches = contacts.search("John")
```

# Extending Behavior with `super()`

## Before (Total Replacement)

- A child method completely `overwrites` the parent method, losing original initialization logic.

## After (Extension via `super()`)

- The child method uses `super()` to trigger the parent's logic first, then safely adds its own.

## Why It Matters

- If we override `__init__` and forget to call parent's `__init__`, the object will **lack its fundamental data**.

## The Solution

- The `super()` function safely delegates method calls up to the parent class.

# Using super() in Initialization

## 1. Complete Replacement (parent logic **lost**):

```
class Friend(Contact):  
    def __init__(self, name, email, phone):  
        self.phone = phone    # name, email never set!
```

## 2. Extension via super().\_\_init\_\_ (parent logic **preserved**):

```
class Friend(Contact):  
    def __init__(self, name, email, phone):  
        super().__init__(name, email)    # parent init  
        self.phone = phone                # child-specific
```

## 3. Extension via super().method() (same pattern for any method):

```
class Friend(Contact):  
    def info(self):  
        base = super().info()            # parent logic first  
        return f"{base} | Phone: {self.phone}"    # extend
```

# Multiple Inheritance Architecture

## Definition

- Python supports **multiple inheritance**, allowing a class to inherit attributes and methods from more than one parent class.

## Example

- **Contact** (provides name, email) + **AddressHolder** (provides street, city)  
= **Friend** (inherits all 4 attributes)

## Caution

- Introduces significant complexity regarding **initialization and method resolution**.

# Implementing Multiple Inheritance

## Code Snippet:

```
class AddressHolder:
    def __init__(self, street: str, city: str):
        self.street = street
        self.city = city

# Inheriting from both Contact and AddressHolder
class Friend(Contact, AddressHolder):
    def __init__(self, name, email, phone, street, city):
        # Naive approach: explicitly calling both parents
        Contact.__init__(self, name, email)
        AddressHolder.__init__(self, street, city)
        self.phone = phone
```

# The Diamond Problem

## The Diamond Shape

- BaseClass at the top, LeftSubclass and RightSubclass in the middle, Subclass at bottom.

## ✗ The Pitfall

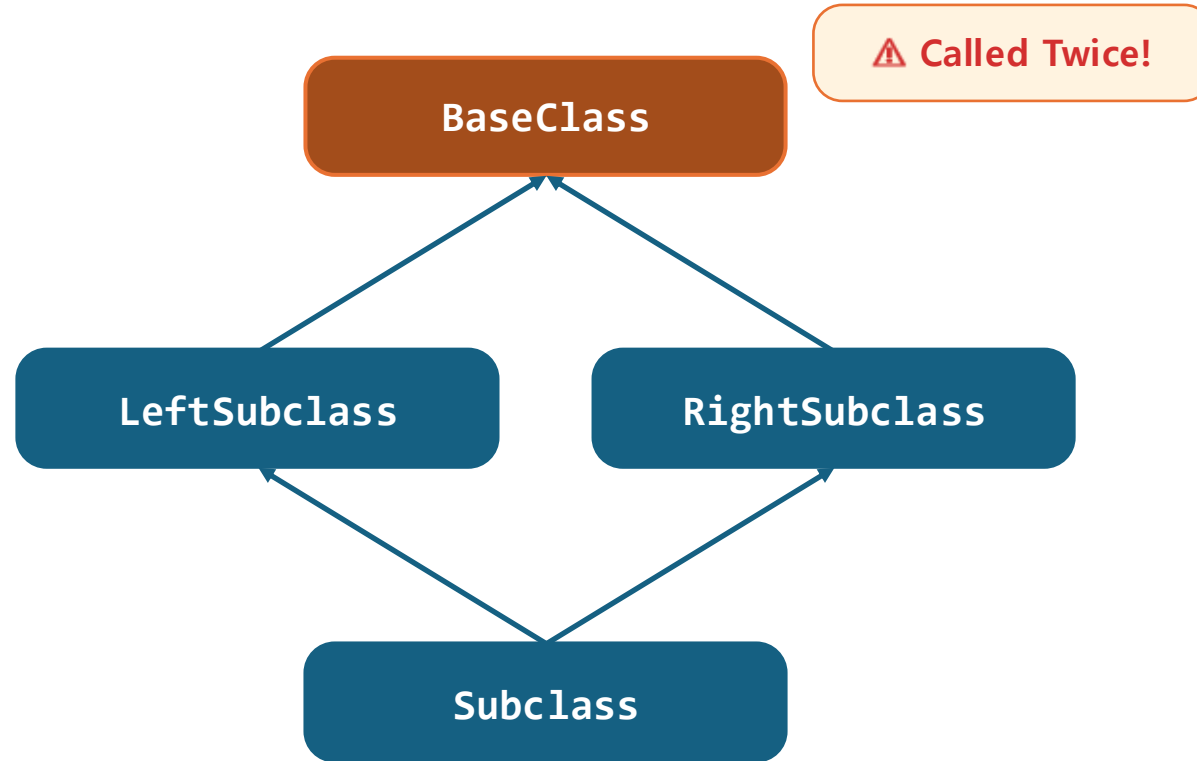
- When a class inherits from two classes that share a common ancestor, naive initialization causes the top-level ancestor's `__init__` to be executed **twice**.

## Consequences

- Can overwrite data, duplicate database connections, or cause performance bottlenecks.

# The Diamond Problem

## Class Hierarchy



→ Next: MRO + `super()` solves this!

## ✗ Naive `__init__` Calls

- 1 `Subclass.__init__()`
- 2 `LeftSubclass.__init__()`
- 3 `BaseClass.__init__()` ← 1st call
- 4 `RightSubclass.__init__()`
- 5 `BaseClass.__init__()` ← 2nd call ⚠

⚠ **Result: `BaseClass.__init__()` runs twice!**  
Data overwritten, duplicated resources,  
performance bottlenecks

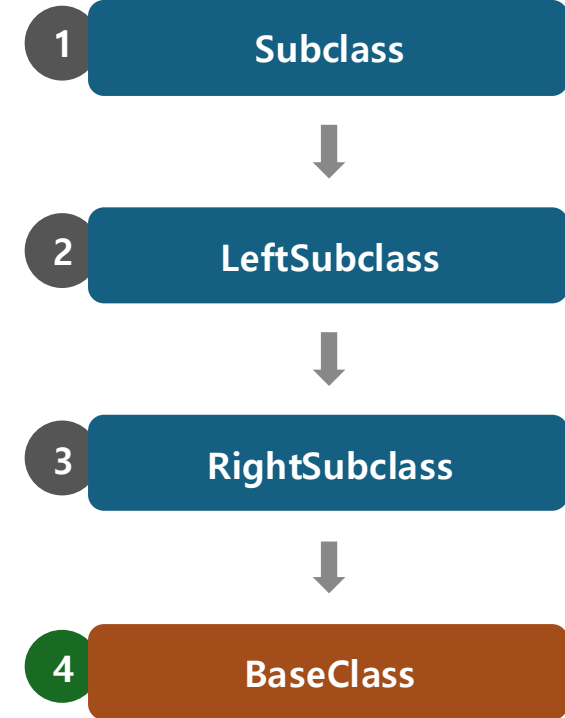
# MRO + super() to the Rescue

## Code Snippet:

```
class BaseClass:
    num_base_calls = 0
    def call_me(self):
        BaseClass.num_base_calls += 1
class LeftSubclass(BaseClass):
    def call_me(self):
        super().call_me()
class RightSubclass(BaseClass):
    def call_me(self):
        super().call_me()
class Subclass(LeftSubclass, RightSubclass):
    def call_me(self):
        super().call_me()

s = Subclass(); s.call_me()
print(s.num_base_calls)
# Output: 1 (MRO sets order;
#          super() follows it)
```

## MRO Call Order



✓ BaseClass called only once!

# Passing Arguments in MRO

## Step 1

- `super()` passes control along the MRO chain.

## Step 2

- Different classes need different initialization arguments.

## Step 3

- Solution: Use `**kwargs` to collect all arguments as a dictionary.
- Pass everything as keywords at creation:  

```
Subclass(name="Alice", email="a@b.com",  
         street="123 Main", city="Seoul")
```
- All packed into one `kwargs` dict → each class picks what it needs.

## Step 4

- Each class extracts its required keys from `**kwargs` and passes the remaining to the next class via `super()`.

# Cooperative `__init__` with `**kwargs`

## Code Snippet:

```
class Contact:
    def __init__(self, name="", email="", **kwargs):
        super().__init__(**kwargs)
        self.name, self.email = name, email

class AddressHolder:
    def __init__(self, street="", **kwargs):
        super().__init__(**kwargs)
        self.street = street

class Friend(Contact, AddressHolder):
    def __init__(self, phone="", **kwargs):
        super().__init__(**kwargs)
        self.phone = phone
```

# Polymorphism & Duck Typing

*"If it walks like a duck and quacks like a duck, it's a duck."*

Polymorphism means "many forms".

It allows different classes to share a common method name.

In Python, "Duck Typing" means the language cares about an object's **behavior** (its methods), not its strict inheritance lineage.

# Polymorphism in Action

## Code Snippet:

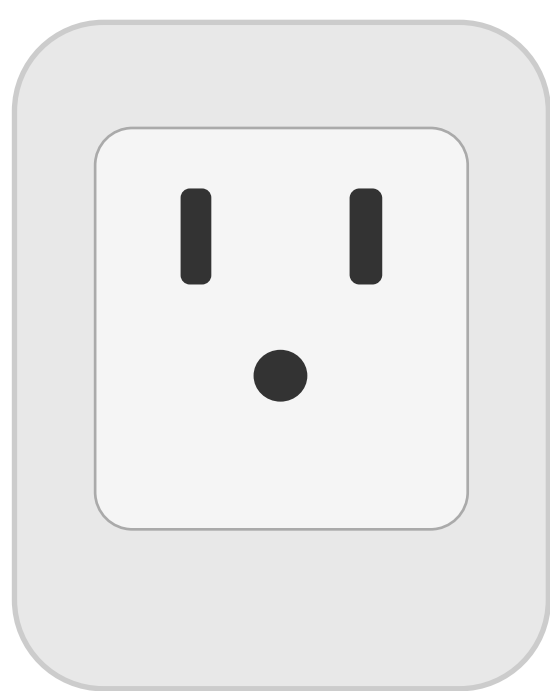
```
class MP3File:
    def play(self):
        print("Playing MP3...")

class FlacFile:
    def play(self):
        print("Playing FLAC losslessly...")

# This function relies on Duck Typing
def play_media(media_object):
    media_object.play()

# No shared base class, but both work!
play_media(MP3File())
play_media(FlacFile())
```

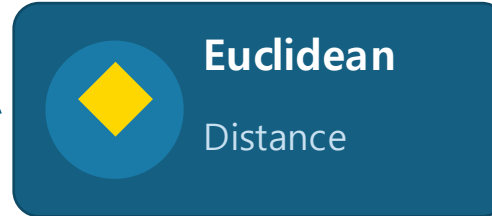
# Polymorphism = Power Outlet



## The Interface

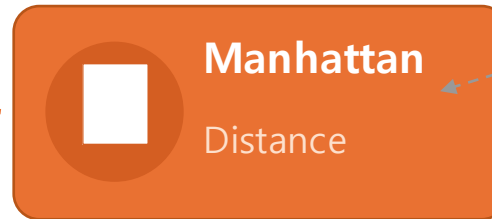
`compute(s1, s2) → float`

`.compute()`



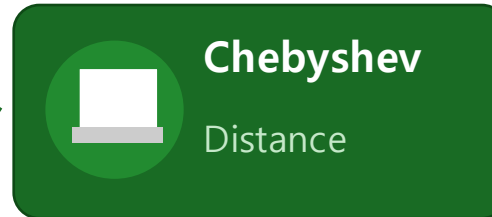
Euclidean  
Distance

`.compute()`



Manhattan  
Distance

`.compute()`



Chebyshev  
Distance

*Different devices, same plug shape  
= Different subclasses, same method*

## KNN Classifier

```
self.distance.compute(a, b)
```

*"I don't care which — just compute!"*



**Power Outlet Analogy:** The outlet doesn't care if you plug in a lamp, charger, or laptop — as long as the plug fits (the interface), electricity flows. Similarly, KNN doesn't care **which** Distance — just that it has `compute()`

# Case Study – KNN Distance

## Hierarchy Tree

- Top node: Hyperparameter (contains algorithm config)
- Points to Abstract node: Distance Base Class

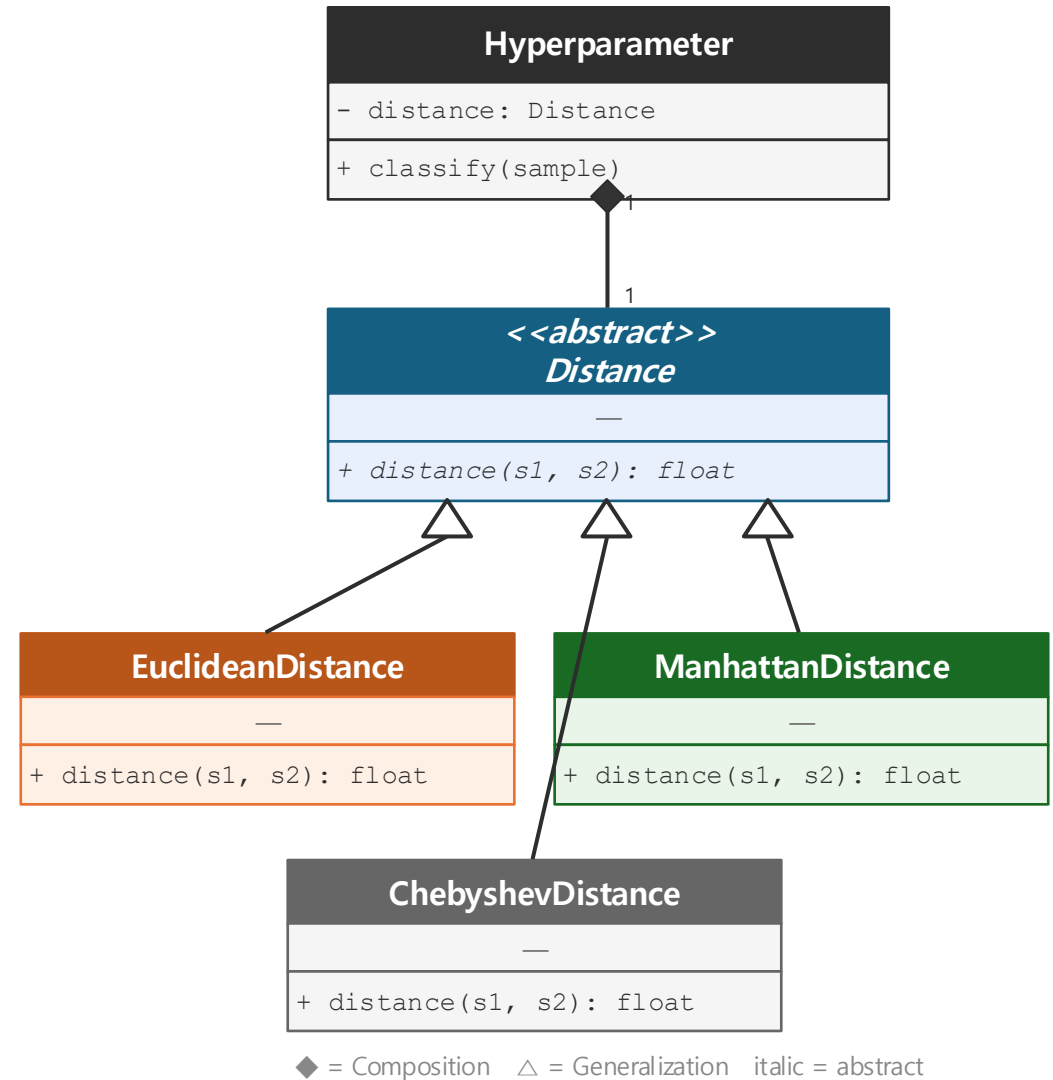
## Concrete Implementations

- EuclideanDistance
- ManhattanDistance
- ChebyshevDistance

## Key Insight

- Polymorphism ensures the correct math is applied dynamically.

UML Class Diagram



# Designing the Distance Abstraction

## The Goal

- Calculate the distance between two Sample vectors.

## The Contract

- Every distance class must provide a distance(s1, s2) method.

## The Benefit

- By standardizing the interface, the main ML algorithm can swap distance metrics without changing its core logic (Open/Closed Principle).

# Distance Hierarchy

## Abstract Base Class

- *Distance* with method: + `distance(s1: Sample, s2: Sample): float`

## Concrete Classes

- `EuclideanDistance` (same method signature)
- `ManhattanDistance` (same method signature)

## Liskov Substitution Principle

- Any subclass can **seamlessly replace** the parent class in the application.



Barbara Liskov

# Defining the Abstract Base Class

## Code Snippet:

```
import abc

class Distance(abc.ABC):
    """Abstract base class for distance calculations."""

    @abc.abstractmethod
    def distance(self, s1, s2) -> float:
        pass
```

# Distance Algorithms Compared

## Euclidean Distance

- Standard straight-line distance ("as the crow flies").
- Uses Pythagorean theorem. Excellent for continuous data.

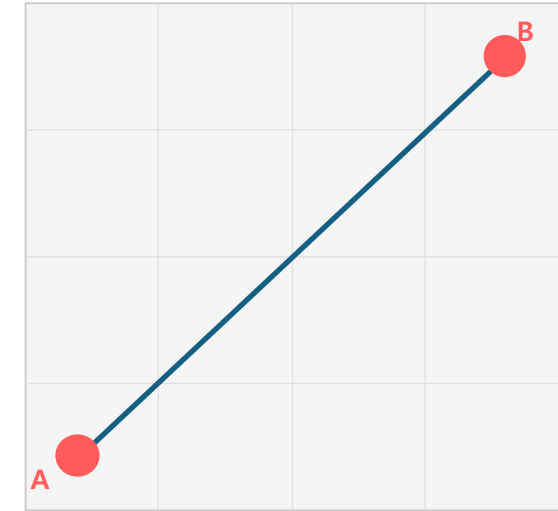
## Manhattan Distance

- Grid-based distance (like navigating city blocks).
- Sum of absolute differences. Often superior for high-dimensional spaces.

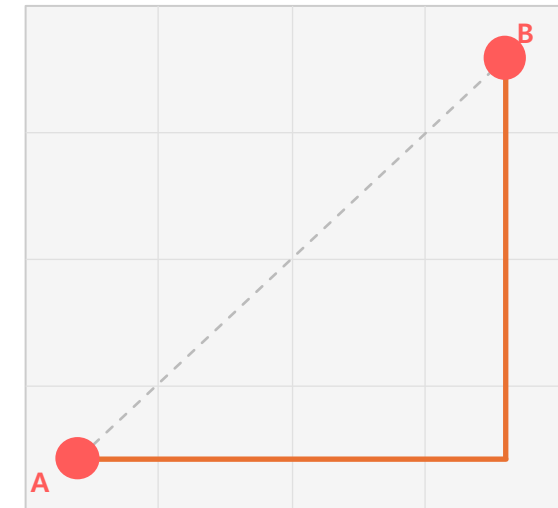
## Key Insight

- Our polymorphic design allows data scientists to **test both instantly**.

Euclidean



Manhattan



# Euclidean vs. Manhattan

## Visual Comparison

- Two points A and B on a graph.

## Euclidean

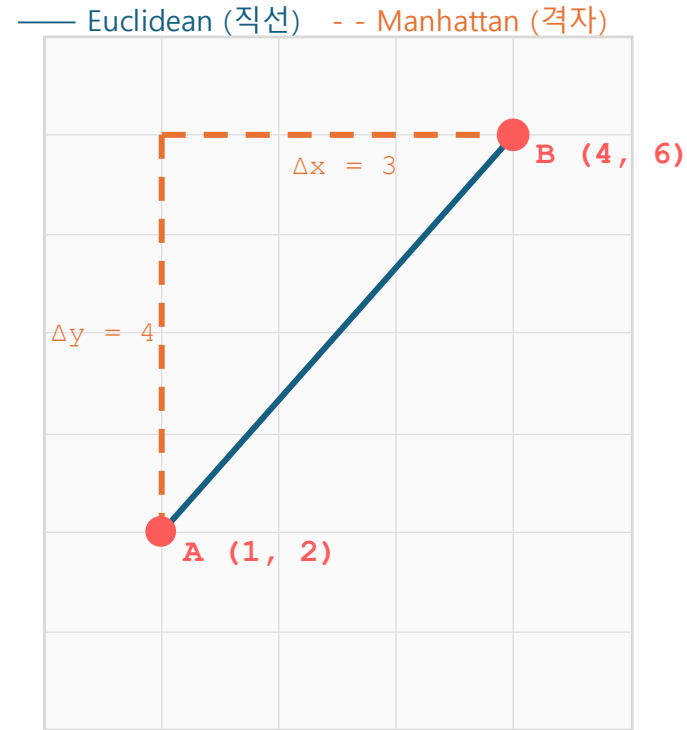
- Diagonal straight line (hypotenuse).

## Manhattan

- Stair-step path along grid (sum of legs).

## Design Principle

- Each algorithm in its own class → math **cleanly separated** from KNN logic.



$$\begin{aligned}\text{Euclidean} \\ & \sqrt{3^2 + 4^2} \\ & = \sqrt{9+16} = \sqrt{25} \\ & = 5.0\end{aligned}$$

$$\begin{aligned}\text{Manhattan} \\ & |4-1| + |6-2| \\ & = 3 + 4 \\ & = 7.0\end{aligned}$$

# From Distance to Norms

## 1. The $L_p$ Norm Unifying Formula

$$\|x\|_p = (\sum |x_i|^p)^{1/p}$$

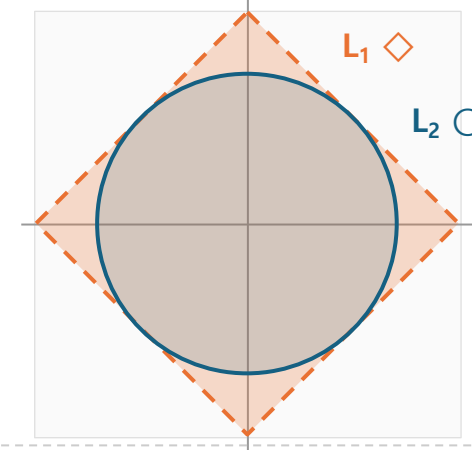
### $L_1$ Norm ( $p=1$ )

= Manhattan Distance  
 $\sum |x_i - y_i|$

### $L_2$ Norm ( $p=2$ )

= Euclidean Distance  
 $\sqrt{(\sum |x_i - y_i|^2)}$

## Unit Ball Shape $\|x\|_p = 1$



## 2. Deep Learning Applications Loss & Regularization

### $L_1$ in Deep Learning

- Loss: MAE (Mean Absolute Error)
- Reg: Lasso → pushes weights to 0 (Sparsity)
- $\diamond$  shape → feature selection effect

### $L_2$ in Deep Learning

- Loss: MSE (Mean Squared Error)
- Reg: Ridge / Weight Decay → shrinks all
- $\circ$  shape → smooth gradient for backprop

 **Key Insight:** The simple distance algorithms you learned today become the  $L_1/L_2$  Norms powering Loss Functions & Regularization in Deep Learning.

# Implementing Concrete Distances

**Code Snippet:** (Simplified 2D example using sepal features only)

```
import math

class EuclideanDistance(Distance):
    def distance(self, s1, s2) -> float:
        # Simplified: 2 of 4 features for clarity
        return math.hypot(
            s1.sepal_length - s2.sepal_length,
            s1.sepal_width - s2.sepal_width)

class ManhattanDistance(Distance):
    def distance(self, s1, s2) -> float:
        # Simplified: 2 of 4 features for clarity
        return (abs(s1.sepal_length - s2.sepal_length) +
                abs(s1.sepal_width - s2.sepal_width))s
```

# Recall

## Reflection Questions

**Q1.** How does the Method Resolution Order (MRO) solve the Diamond Problem?

**Q2.** Why is `super()` essential in complex class hierarchies?

**Q3.** How does "Duck Typing" provide polymorphism without strict inheritance?

**Q1 Hint: Diamond**

C3 linearization  
→ each class once  
→ left-to-right order

**Q2 Hint: `super()`**

Follows MRO, not parent class  
→ cooperative `**kwargs`

**Q3 Hint: Duck Typing**

No `isinstance()`  
→ if it has `.play()`  
it's playable

# Practical Exercises

## ☆☆☆ Exercise 1: Refactor Legacy Code

- Find a script with heavy `if/elif` type-checking logic
- Replace with a `polymorphic class structure` (ABC + concrete subclasses)

Hint: Start by identifying which behavior varies per type

## ★★☆ Exercise 2: Trace the MRO

- Build a 4-level diamond hierarchy with cooperative `super().__init__(**kwargs)`
- Add `print()` in each `__init__` to trace execution order

Hint: Verify with `ClassName.__mro__` or `ClassName.mro()`

## ★★★ Exercise 3: Expand the Case Study

- Implement `ChebyshevDistance` → `max(|xi - yi|)`
- Plug it into KNN without changing the classifier code (OCP!)

Hint: `max(abs(a - b) for a, b in zip(s1, s2))`

# Summary

1

## Inheritance

- ✓ Is-A relationship
- ✓ super() + MRO
- ✓ Diamond Problem
- ✓ Cooperative \*\*kwargs

2

## Polymorphism

- ✓ Duck Typing (no ABC)
- ✓ ABC + @abstractmethod
- ✓ Strategy Pattern
- ✓ Open/Closed Principle

3

## Case Study

- ✓ Distance hierarchy (ABC)
- ✓ Euclidean Distance
- ✓ Manhattan Distance
- ✓ KNN + Strategy Pattern

**Inheritance & Polymorphism are the engines of OOP code reuse.**

We create software that is **modular, scalable, and easy to maintain.**